



MERRIMACK COLLEGE

CSC 6303

Week 8

Multi-thread, parallelization, and scheduling

Systems and Languages Survey - Dr. Paulo Fernandes

Presentation Agenda

Week 8

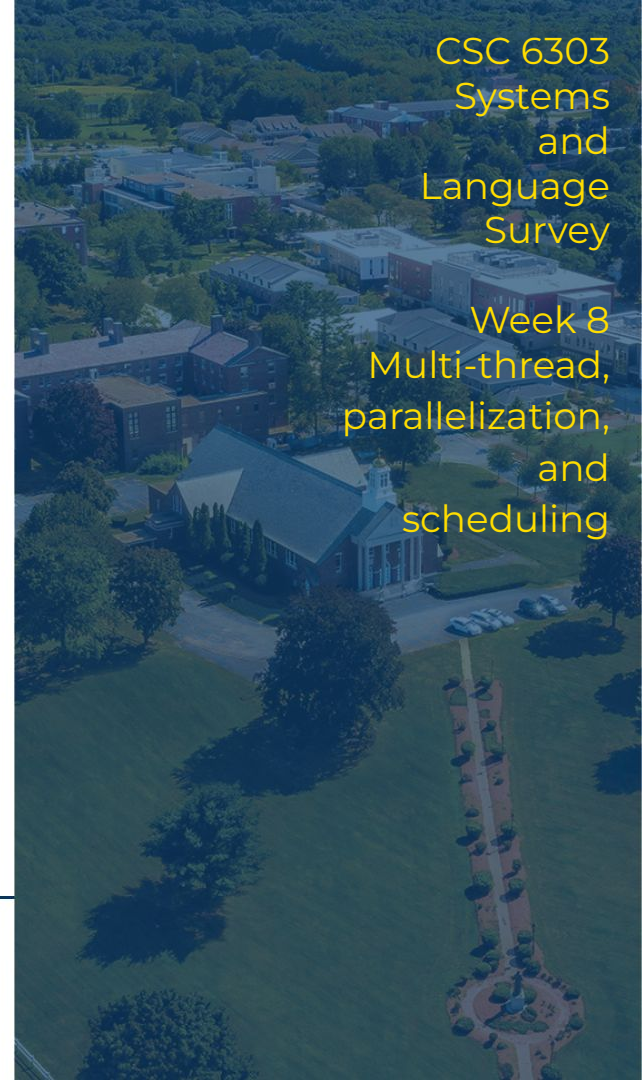
- Multi-thread and parallelization
 - a. Concurrency
 - b. Parallelism
- Scheduling
 - a. Job Scheduler
- This Week Only Task



MERRIMACK COLLEGE

CSC 6303
Systems
and
Language
Survey

Week 8
Multi-thread,
parallelization,
and
scheduling



Multi-thread and parallelization

n An Operating System does more than just pass information along.

Even before multi-core processors, there was the concept of tasks in a program that could be done in parallel, or at least, that the order of execution is not relevant.

This idea was introduced by the 70's with the concept of **concurrency**, but, of course, it gained much more momentum with the multi-core architectures that made easier to transform concurrency into **parallelism**.



Concurrency 101

Let's see the Python implementation to compute the factorial of n implemented as a brute force algorithm.

```
def factorial(n):  
    ans = 1  
    for i in range(n):  
        ans *= i+1  
    return ans
```



The result of each iteration can be computed only after the previous one.

Let's see the Python implementation to multiply two matrices **A** and **B** (assuming compatible dimensions).

```
def matrix_multiply(A, B):  
    n = len(A)  
    m = len(B[0])  
    ans = [[0]*m for _ in range(n)]  
    for i in range(n):  
        for j in range(m):  
            for k in range(len(B)):  
                ans[i][j] += A[i][k] * B[k][j]  
    return ans
```



Each of the elements of ***ans[i][j]*** could have been computed in any order, or even in parallel.



Concurrency 101

Here the tasks that can be dealt with concurrently are pretty simple (the **for k** loop), but in other problems these concurrent candidate tasks could be more complex, deal with different sources or destinations for data that would be useful to make them be executed in any order.

```
def matrix_multiply(A, B):  
    n = len(A)  
    m = len(B[0])  
    ans = [[0]*m for _ in range(n)]  
    for i in range(n):  
        for j in range(m):  
            for k in range(len(B)):  
                ans[i][j] += A[i][k] * B[k][j]  
    return ans
```

For example, let's consider that the origin of the elements of **A** and **B** comes not from local variables, but are transmitted from a remote data server that may sometimes deliver the elements in quite different times. In such case it would be interesting to start all **n** times **m** tasks concurrently and finishing them in the order the data from **A** and **B** arrives.



Advantages of Concurrency

- For I/O bound programs
 - It is important to realize how slow I/O is compared to the CPU processing tasks, even when we are talking of just writing on a file
 - The advantage for I/O bound programs is to overlap the I/O waiting times.
- For CPU bound programs
 - It is important to keep in mind that your program is just one among several programs being dispatched by the OS
 - The advantage for CPU bound programs is to create more threads of execution, thus becoming more competitive for CPU execution slots, which is even better in a multi-core machine.

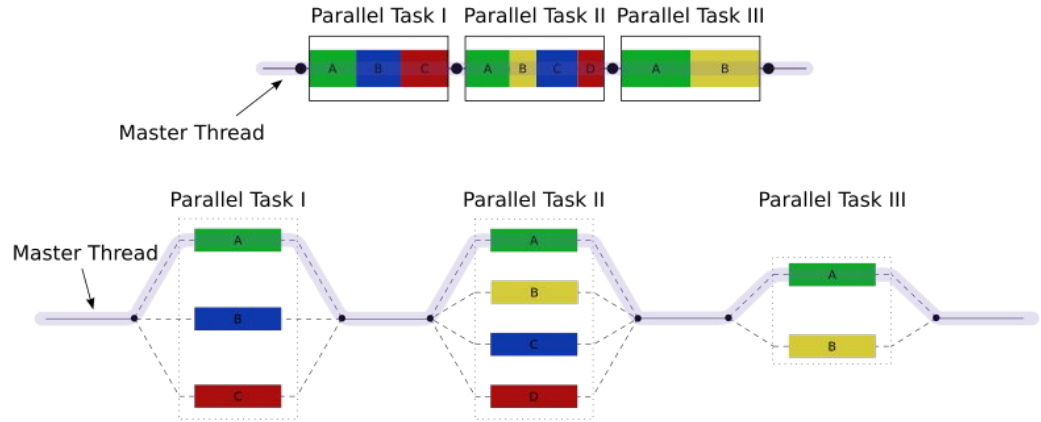


Parallelism

Once you have your program designed with tasks that can be executed in any order, it is just a matter of availability of hardware to make it work in parallel.

However, in parallel programs we tend to focus a little bit more on what can be parallelized (***fork***), and what needs to be synchronized (***join***).

With one processor you have concurrency.
With more than one you have parallelism.



Parallelism in Python

One possible way is to use the package ***multiprocessing*** (***mp***).

The method ***Pool(mp.cpu_count())*** delivers a pool of processors to map the threads upon.

The method ***map*** over the pool of processors deploys the threads (*fork*) with the function name and parameters.

When this command is over (*join*) the variable *result* holds the list of outputs.

```
import multiprocessing as mp

def square(n):
    return n * n

if __name__ == '__main__':
    numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    with mp.Pool(mp.cpu_count()) as p:
        result = p.map(square, numbers)
    print(result)
```



Parallelism in Python

Applying this idea to the matrix multiplication:

- create a function to perform each task: **_compute_element**;
- create a list of tasks' parameters: **tasks**;
- start the parallel tasks: **mp.Pool** (fork);
- fetch the results from **results** (join - line 17).

```
1  import multiprocessing as mp
2
3  def _compute_element(args):
4      A, B, i, j = args
5      value = 0
6      for k in range(len(B)):
7          value += A[i][k] * B[k][j]
8      return (i, j, value)
9
10 def matrix_multiply_parallel(A, B):
11     n = len(A)
12     m = len(B[0])
13     ans = [[0] * m for _ in range(n)]
14     tasks = [(A, B, i, j) for i in range(n) for j in range(m)]
15     with mp.Pool(mp.cpu_count()) as pool:
16         results = pool.map(_compute_element, tasks)
17     for i, j, value in results:
18         ans[i][j] = value
19
20     return ans
```



Scheduling

How an Operating System does more than one thing at the time?

The concept of multithreading, concurrency, and even parallelism is at the core of the implementation of Operating System. Since the OS is by its nature a program running processes (other programs) concomitantly.

This is done in terms of the **scheduling of processes** and it also includes OS own tasks.



Scheduling Processes



© TRON Legacy. Disney

All programs running within your machine became a process under the Operating System management through a Job Scheduler.

Each of those process are numbered (PID) and they compete for execution slots, thus becoming "alive", i.e. they execute their code.

```
Processes: 406 total, 2 running, 8 stuck, 396 sleeping, 1711 threads 11:17:27
Load Avg: 2.44, 2.64, 3.39 CPU usage: 12.50% user, 6.36% sys, 81.13% idle
SharedLibs: 178M resident, 22M data, 15M linkedit.
MemRegions: 170158 total, 7295M resident, 93M private, 1650M shared.
PhysMem: 16G used (3713M wired), 15M unused.
VM: 1144G vsize, 528M framework vsize, 52644734(191) swpains, 54958458(0) swapou
Networks: packets: 28613845/236 in, 14022145/2023M out.
Disks: 5067108/280G read, 6930865/489G written.
```

PID	COMMAND	%CPU	TIME	#TH	#WO	#PORT	MEM	PURG	CHPRS	PGRP
37830	com.apple.We	27.7	02:42:23	24	4	317	666M+	87M	855M-	37830
18853	Safari	5.4	05:15:09	11	3	623	155M+	2908K	169M	18853
48517	top	5.3	00:02:15	1/1	0	22	5696K	0B	0B	48517
302	WindowServer	4.7	12:48:57	5	2	1326	99M+	413M	239M	302
821	sysmond	4.3	02:29:40	3	2	35	3908K	0B	4448K	821
104	hidd	3.7	92:25:07	7	3	101	1640K	0B	1004K	104
0	kernel_task	3.6	05:41:48	122/4	0	2	2668M+	0B	0B	0
34952	com.apple.We	3.6	06:48:25	15	2	264	771M-	12M	441M	34952
915	Activity Mon	3.0	45:18:07	7	4	219	27M+	380K	37M+	915
37357	com.apple.We	1.5	12:39:04	17	1	285	154M+	64K	538M-	37357
18868	com.apple.We	1.4	70:53:31	15	0	310	680M	227M	438M	18868
572	Spotlight	1.0	13:39:51	10	1	487	8196K	0B	52M	572
40240	RecordIt	1.0	00:01:63	11	4	223	32M	16M	2216K	40240
18864	com.apple.We	0.8	01:20:73	22	6	335	355M-	176M	365M	18864

In multi-core machines you have an specific processing line to each core and the processes are allocated to them.

In Unix-like system you can check these processing line status with the command **top** (at the terminal).



MERRIMACK COLLEGE

Wikipedia: [Job Scheduler](https://en.wikipedia.org/wiki/Job_scheduler).

Checking the scheduler

Besides the live observation of the scheduling lines, both bash and PowerShell offer the **ps** command to check out the current processes owned by the user.

Once you know the PID of your processes you take actions as removing them, changing their priority level, etc.

```
Processes: 406 total, 2 running, 8 stuck, 396 sleeping, 1711 threads 11:17:27
Load Avg: 2.44, 2.64, 3.39 CPU usage: 12.50% user, 6.36% sys, 81.13% idle
SharedLibs: 178M resident, 22M data, 15M linkedit.
MemRegions: 170158 total, 7295M resident, 93M private, 1650M shared.
PhysMem: 16G used (3713M wired), 15M unused.
VM: 1144G vsize, 528M framework vsize, 52644734(191) swapins, 54958458(0) swapou
Networks: packets: 20613845/23G in, 14022145/2023M out.
Disks: 5067188/288G read, 6930865/489G written.
```

PID	COMMAND	%CPU	TIME	#TH	#WQ	#PORT	MEM	PURG	CMPRS	PGRP
37830	com.apple.We	27.7	02:42:23	24	4	317	666M-	87M	855M-	37830
18853	Safari	5.4	05:15:09	11	3	623	155M+	2908K	169M	18853
40517	top	5.3	00:02:15	1/1	0	22	5696K	0B	0B	40517
302	WindowServer	4.7	12:48:57	5	2	1326	99M+	413M	239M	302
821	sysmond	4.3	02:29:40	3	2	35	3908K	0B	4448K	821
104	hidd	3.7	92:25:07	7	3	101	1640K	0B	1004K	104
0	kernel_task	3.6	05:41:48	122/4	0	2	2068M+	0B	0B	0
34952	com.apple.We	3.6	86:48:25	15	2	264	771M-	12M	441M	34952
915	Activity Mon	3.0	45:18:07	7	4	219	27M+	380K-	37M+	915
37357	com.apple.We	1.5	12:39:04	17	1	285	154M+	64K	530M-	37357
18868	com.apple.We	1.4	70:53:31	15	0	310	680M	227M	438M	18868
572	Spotlight	1.0	13:39:51	10	1	487	8196K	0B	52M	572
40240	RecordIt	1.0	00:01:63	11	4	223	32M	16M	2216K	40240
18864	com.apple.We	0.8	61:20:73	22	6	335	355M-	176M	365M	18864

Using multithreading, you just might need to act on your processes.



This Week's Only Task

- Final Exam

The Final Exam is to be taken online

- It will be available this Wednesday after the live session (9 PM EST);
- It is to be taken within 4 hours after starting;
- The final hard deadline for the Final Exam is Saturday 11:59 PM EST.



Final Exam

- This Wednesday at 9 PM EST the exam will be available on Canvas;
- The exam is to be taken online within 4 hours counting as you start it;
- The exam has to be submitted no later than this Saturday 11:59 PM EST;
- The format for the submission is a single pdf file that may include typed textual files, print screens, photos of handwritten materials, etc.;
- The exam has 8 questions, one related to each topic, try not to spend more than 30 minutes to each question;
- The exam is open book, open notes, but you are not allowed to access the general search within the Internet.



MERRIMACK COLLEGE

The Final Exam Grade counts as
25% of the course overall grade.

” **Check out recommended texts and videos**

- **Make sure your Week 7 tasks are done**
- **Take the final exam no later than this Saturday**

**It was my honor and privilege to be your instructor.
Thank you!!**



MERRIMACK COLLEGE